

Unit-4

Exception Handling..

Level A: Easy Questions (2 Marks)

Q1. What are byte streams in Java?

Byte streams handle data in binary format (8 bits at a time). They are used to perform input and output (I/O) operations on raw binary data like images or files. Examples include `FileInputStream` and `FileOutputStream`.

Q2. Define character streams in Java.

Character streams handle data as characters (16 bits at a time, supporting Unicode). They are ideal for text data and include classes like `FileReader` and `FileWriter`.

Q3. What is the purpose of the File class in Java?

The `File` class represents file and directory pathnames. It provides methods to create, delete, and check the existence of files or directories.

Q4. Explain the use of FileInputStream in Java.

`FileInputStream` is used to read raw bytes from a file. It's commonly used for reading binary data like images or videos.

Q5. What is FileOutputStream, and how is it used?

`FileOutputStream` writes raw bytes to a file. It's used to create and write data to binary files.

Q6. Define the InputStreamReader class in Java.

`InputStreamReader` converts byte streams to character streams, enabling text-based processing of byte data.

Q7. Explain the purpose of OutputStreamWriter in Java.

`OutputStreamWriter` converts character streams into byte streams, allowing character data to be written to binary files.

Q8. What is the difference between Reader and Writer classes in Java?

- Reader classes are used for reading character data.
- Writer classes are used for writing character data.

Q9. Explain the purpose of the BufferedReader class.

BufferedReader reads text efficiently by buffering characters to reduce I/O operations, improving performance.

Q10. Define the List class in the Java Collection Framework.

The List interface represents an ordered collection that allows duplicate elements. Common implementations include ArrayList and LinkedList.

Q11. What is the difference between ArrayList and LinkedList?

- ArrayList: Backed by an array, faster for random access.
- LinkedList: Uses nodes, better for frequent insertions/deletions.

Q12. What are the rules of the ArrayList class?

1. Allows duplicate elements.
2. Maintains insertion order.
3. Supports dynamic resizing.

Q13. Explain the difference between doubly and circular linked lists.

- **Doubly linked list:** Each node has pointers to the next and previous nodes.
- **Circular linked list:** The last node points back to the first node.

(Diagram will be provided if required.)

Q14. Define the Vector class in Java and its uses.

Vector is a synchronized list class, similar to ArrayList. It supports dynamic resizing and is thread-safe.

Q15. What is the purpose of the write() method in file handling?

The write() method writes data to a file or output stream.

Q16. How does the read() method work in file handling?

The read() method reads a single byte or character from an input stream or file.

Q17. What are the steps of Java file handling?

1. Create or open a file.
2. Perform read/write operations.
3. Close the file.

Q18. What are legacy classes in Java?

Legacy classes (e.g., Vector, Stack, Hashtable) existed before Java 2 and are synchronized but less commonly used today.

Q19. What is the purpose of the close() method in file handling?

The close() method releases system resources associated with a file or stream.

Q20. How can you check if a file exists in Java?

Use the exists() method of the File class:

java

Copy code

```
File file = new File("path/to/file");  
if (file.exists()) {  
    System.out.println("File exists");  
}
```

Level B: Intermediate Questions (5 Marks)

Q21. Describe at least five differences between byte streams and character streams.

Aspect	Byte Streams	Character Streams
Data Type	Handles binary data (bytes).	Handles text data (characters).
Classes	InputStream, OutputStream	Reader, Writer
Usage	For non-text files (images).	For text files (documents).
Size	8-bit data processing.	16-bit data processing.
Encoding	No automatic encoding.	Supports character encoding.

Q22. Write a program in Java to read data from a text file using FileReader.

```
import java.io.*;  
  
public class ReadFileExample {  
    public static void main(String[] args) {  
        try (FileReader reader = new FileReader("example.txt")) {  
            int character;  
            while ((character = reader.read()) != -1) {
```

```
        System.out.print((char) character);
    }
} catch (IOException e) {
    e.printStackTrace();
}
}
}
```

Q23. Write a program in Java to write data to a text file using FileWriter.

```
import java.io.*;

public class WriteFileExample {
    public static void main(String[] args) {
        try (FileWriter writer = new FileWriter("output.txt")) {
            writer.write("Hello, this is a test file!");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Q24. Write a Java program to demonstrate the use of BufferedReader.

```
import java.io.*;

public class BufferedReaderExample {
    public static void main(String[] args) {
        try (BufferedReader br = new BufferedReader(new FileReader("example.txt"))) {
            String line;
            while ((line = br.readLine()) != null) {

```

```
        System.out.println(line);
    }
} catch (IOException e) {
    e.printStackTrace();
}
}
}
```

Q25. Explain the use of FileInputStream with examples.

FileInputStream is used to read raw bytes from a file, often for non-text files like images or videos.
Example:

```
import java.io.*;

public class FileInputStreamExample {
    public static void main(String[] args) {
        try (FileInputStream fis = new FileInputStream("example.txt")) {
            int byteData;
            while ((byteData = fis.read()) != -1) {
                System.out.print((char) byteData);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Q26. Write a Java program to implement ArrayList and demonstrate adding, removing, and adding all elements.

```
import java.util.*;
```

```

public class ArrayListExample {

    public static void main(String[] args) {

        ArrayList<String> list = new ArrayList<>();

        list.add("Apple");

        list.add("Banana");


        System.out.println("List: " + list);


        list.remove("Apple");

        System.out.println("After removal: " + list);


        ArrayList<String> newList = new ArrayList<>();

        newList.add("Cherry");

        newList.add("Date");


        list.addAll(newList);

        System.out.println("After adding all: " + list);

    }

}

```

Q27. Describe the differences between Vector and LinkedList with examples.

- Vector is synchronized, while LinkedList is not.
- Vector uses a dynamic array; LinkedList uses nodes.
- Example usage:

java

Copy code

```
import java.util.*;
```

```

public class VectorLinkedListExample {

    public static void main(String[] args) {

        Vector<String> vector = new Vector<>();

        vector.add("A");

        vector.add("B");

        System.out.println("Vector: " + vector);


        LinkedList<String> linkedList = new LinkedList<>();

        linkedList.add("X");

        linkedList.add("Y");

        System.out.println("LinkedList: " + linkedList);

    }

}

```

Q28. Write a program to demonstrate the use of ArrayList with its 5 methods.

```

import java.util.*;

public class ArrayListMethodsExample {

    public static void main(String[] args) {

        ArrayList<Integer> list = new ArrayList<>();

        list.add(10);

        list.add(20);

        list.add(30);

        System.out.println("Initial List: " + list);


        list.add(1, 15); // add at index

        System.out.println("After adding 15: " + list);


        list.remove(2); // remove at index
    }

}

```

```
System.out.println("After removing element: " + list);

list.set(1, 25); // update value

System.out.println("After setting value: " + list);

System.out.println("Contains 20? " + list.contains(20));
}
}
```

Q29. Write a program to demonstrate the use of heterogeneous data in ArrayList.

```
import java.util.*;

public class HeterogeneousArrayList {
    public static void main(String[] args) {
        ArrayList<Object> list = new ArrayList<>();
        list.add("String");
        list.add(123);
        list.add(45.6);
        list.add(true);

        System.out.println("Heterogeneous List: " + list);
    }
}
```

Q30. Write a program to demonstrate the use of the List interface.

```
import java.util.*;

public class ListInterfaceExample {
    public static void main(String[] args) {
```



```
List<String> list = new ArrayList<>();

list.add("Java");

list.add("Python");

list.add("C++");


for (String lang : list) {

    System.out.println(lang);

}

}

public static void main(String[] args) {

    String fileName = "example.txt";

    try (FileWriter writer = new FileWriter(fileName)) {

        writer.write("Hello, world!");

    } catch (IOException e) {

        e.printStackTrace();

    }


    try (BufferedReader reader = new BufferedReader(new FileReader(fileName))) {

        String line;

        while ((line = reader.readLine()) != null) {

            System.out.println(line);

        }

    } catch (IOException e) {

        e.printStackTrace();

    }

}

}
```

Level C: Difficult Questions (10 Marks)

Q31. Write a Java program to create a text file, write data to it, then read and display the content.

java

Copy code

```
import java.io.*;

public class FileReadWriteExample {

    public static void main(String[] args) {

        String fileName = "example.txt";

        try (FileWriter writer = new FileWriter(fileName)) {

            writer.write("Hello, world!");

        } catch (IOException e) {

            e.printStackTrace();

        }

        try (BufferedReader reader = new BufferedReader(new FileReader(fileName))) {

            String line;

            while ((line = reader.readLine()) != null) {

                System.out.println(line);

            }

        } catch (IOException e) {

            e.printStackTrace();

        }

    }

}
```

Q32. Write a Java program using FileOutputStream with examples.

java

Copy code

```
import java.io.*;
```

```
public class FileOutputStreamExample {  
    public static void main(String[] args) {  
        String data = "This is an example of FileOutputStream.";  
        try (FileOutputStream fos = new FileOutputStream("output.txt")) {  
            byte[] byteData = data.getBytes();  
            fos.write(byteData);  
            System.out.println("Data written to file successfully.");  
        } catch (IOException e) {  
            e.printStackTrace();  
        }  
    }  
}
```

Q33. a) Explain the hierarchy of Java I/O classes with a flowchart.

Hierarchy of Java I/O Classes:

- **Root Classes:**
 - InputStream (for reading byte data)
 - OutputStream (for writing byte data)
 - Reader (for reading character data)
 - Writer (for writing character data)

Byte Streams:

- FileInputStream (read from files)
- FileOutputStream (write to files)

Character Streams:

- FileReader (read text files)

- `FileWriter` (write text files)

Buffered Streams:

- `BufferedReader` (read with buffering)
- `BufferedWriter` (write with buffering)

(Flowchart can be shared if needed.)

b) Explain the hierarchy of the collection framework.

Java Collection Framework Hierarchy:

1. Interfaces:

- `Collection`
 - `List` (e.g., `ArrayList`, `LinkedList`, `Vector`)
 - `Set` (e.g., `HashSet`, `LinkedHashSet`, `TreeSet`)
 - `Queue` (e.g., `PriorityQueue`)

2. Classes:

- `ArrayList`, `LinkedList` (`List` implementations)
 - `HashSet`, `TreeSet` (`Set` implementations)
 - `HashMap`, `TreeMap` (`Map` implementations)
-

Q34. Explain the structure and features of the `LinkedList` class in Java's Collection Framework.

Features of `LinkedList`:

1. Implements the `List` and `Deque` interfaces.
2. Doubly-linked node structure for efficient insertions and deletions.
3. Allows null values and duplicates.
4. Supports operations such as adding elements at the head or tail.

Program to Demonstrate `LinkedList` Operations:

```
java
```

```
Copy code
```

```
import java.util.LinkedList;
```

```
public class LinkedListExample {  
    public static void main(String[] args) {  
        LinkedList<String> list = new LinkedList<>();  
  
        // Adding elements  
        list.add("Apple");  
        list.add("Banana");  
        list.add("Cherry");  
        System.out.println("LinkedList: " + list);  
  
        // Getting elements  
        System.out.println("First element: " + list.get(0));  
  
        // Clearing the list  
        list.clear();  
        System.out.println("After clearing: " + list);  
    }  
}
```

Q35. Discuss the Collection Framework in Java. Write a program to demonstrate ArrayList, LinkedList, and Vector.

Explanation:

The Java Collection Framework provides standard classes and interfaces to store and manipulate groups of objects efficiently.

Program:

java

Copy code

```
import java.util.*;
```

```

public class CollectionFrameworkExample {

    public static void main(String[] args) {

        // ArrayList

        ArrayList<Integer> arrayList = new ArrayList<>();

        arrayList.add(10);

        arrayList.add(20);

        System.out.println("ArrayList: " + arrayList);


        // LinkedList

        LinkedList<Integer> linkedList = new LinkedList<>();

        linkedList.add(30);

        linkedList.add(40);

        System.out.println("LinkedList: " + linkedList);


        // Vector

        Vector<Integer> vector = new Vector<>();

        vector.add(50);

        vector.add(60);

        System.out.println("Vector: " + vector);

    }

}

```

Q36. a) Explain File, FileInputStream, and FileOutputStream classes in Java.

1. **File Class:** Represents file or directory pathnames. Used to create, delete, or check file properties.
2. **FileInputStream Class:** Reads raw bytes from files. Useful for binary data like images.
3. **FileOutputStream Class:** Writes raw bytes to files. Ideal for binary data output.

Usage Example for All:

java

Copy code

```
import java.io.*;
```

```
public class FileExample {  
    public static void main(String[] args) {  
        try {  
            File file = new File("example.txt");  
  
            // Create a new file  
            if (file.createNewFile()) {  
                System.out.println("File created: " + file.getName());  
            } else {  
                System.out.println("File already exists.");  
            }  
  
            // Write to the file  
            try (FileOutputStream fos = new FileOutputStream(file)) {  
                fos.write("Hello, File Handling!".getBytes());  
            }  
  
            // Read from the file  
            try (FileInputStream fis = new FileInputStream(file)) {  
                int content;  
                while ((content = fis.read()) != -1) {  
                    System.out.print((char) content);  
                }  
            }  
        } catch (IOException e) {  
            e.printStackTrace();  
        }  
    }  
}
```

```
    }  
}  
}
```

b) Write the main properties of Vector.

1. Thread-safe and synchronized.
2. Allows dynamic resizing.
3. Supports random access with indexing.
4. Can store duplicate and null elements.